Implementation Study of a Probability-Guided Approximate Multiplier for Low-Power Image Processing

Naresh Godara, Siddharth Khandelwal, Anushka Mishra, Rishu Kumar
Department of Electronics and Communication Engineering
Indian Institute of Information Technology, Nagpur

ABSTRACT

Hardware designers targeting low-power embedded systems increasingly adopt intentional arithmetic approximation to achieve efficiency gains that exact logic cannot deliver. This work undertakes a systematic reproduction of the probability-guided approximate 4:2 compressor architecture proposed in the recent literature, in which a combinational input-sorting network reduces the effective compressor state-space from sixteen patterns to six canonical classes and concentrates any introduced errors onto statistically infrequent input combinations. Two compressor variants — one accuracy-prioritized and one hardware-minimized — are individually embedded into an 8×8 Dadda multiplier synthesized under 32 nm CMOS technology at 0.85 V. Exhaustive simulation across all 65,536 input combinations confirms the headline results: an average Power Delay Product reduction of 20.19% relative to an exact multiplier baseline, and a Mean Structural Similarity Index Measure of 97.59% across image multiplication, smoothing, and sharpening tasks. The paper additionally proposes seven concrete directions through which the architecture may be extended, covering wider operand support, runtime configurability, neural-network deployment, multi-node characterization, error compensation circuitry, formal verification, and broader perceptual benchmarking.

Index Terms — approximate computing, 4:2 compressor, Dadda multiplier, input reordering, MSSIM, Power Delay Product, image processing, low-power design.

I. INTRODUCTION

The unrelenting expansion of edge computing, IoT devices, and on-device machine inference has created intense pressure on hardware designers to simultaneously maximize throughput and minimize energy consumption. Portable and embedded platforms — smartphones, surveillance cameras, medical wearables — execute millions of multiply-accumulate operations per second against a fixed battery capacity and a tight thermal ceiling. Finding architectural strategies that cut power without unacceptable quality loss is therefore a high-priority research objective.

Approximate computing exploits a fundamental property of many real-world workloads: human perception and many algorithmic pipelines tolerate bounded numerical imprecision. A pixel value shifted by a few counts is invisible to the eye; a transformed coefficient perturbed by a small amount does not materially degrade a classification score. This tolerance motivates deliberate introduction of controlled errors into arithmetic circuits in exchange for measurable reductions in area, propagation delay, and switching power [1].

Within the arithmetic datapath, the multiplier is among the most power-hungry elements. Its internal partial-product reduction tree — built from 4:2 compressor cells — dominates both area and dynamic power. The 4:2 compressor collapses four same-weight bits and one carry-in into a sum and two carry outputs; redesigning this cell with approximation is therefore a high-leverage optimization point.

The reference architecture studied here [2] introduces a novel pre-sorting network that rearranges the four compressor inputs into a canonical order before any logic evaluation occurs. This collapses sixteen raw input patterns into six equivalence classes and ensures that approximation errors fall exclusively on patterns with low statistical probability in typical partial-product data. Two compressor variants with different accuracy-hardware trade-offs are defined and embedded into an 8×8 Dadda multiplier, yielding results superior to a broad comparison field.

This paper reproduces these results independently, verifies the key claims, and proposes seven avenues for future research. Our main contributions are:

- Independent derivation and exhaustive verification of both approximate compressor truth tables.
- Reproduction of accuracy and circuit-performance metrics under 32 nm CMOS at 0.85 V.
- Seven concrete future research directions extending the original contribution.

II. RELATED WORK

A. Exact 4:2 Compressor Foundation

A 4:2 compressor accepts five inputs — four primary bits X1–X4 of equal column weight and a carry-in Cin — and produces three outputs: Sum, Carry, and Cout. Implemented as two cascaded full adders, the exact cell satisfies:

$$Sum = X1 \oplus X2 \oplus X3 \oplus X4 \oplus Cin$$

$$Cout = (X1 \oplus X2)X3 + (X1 \oplus X2)`X1$$

$$Carry=(X1\oplus X2\oplus X3\oplus X4)Cin + (\bar{X}1\oplus X2\oplus X3\oplus X4)`X4$$

When compressors chain together in the Dadda tree, Cout of one stage feeds Cin of the next. Both carry outputs carry identical positional weight; their accurate computation is therefore critical to the multiplier's overall precision.

B. Representative Approximate Designs

Prior work attacks the hardware cost of 4:2 compressors from several angles. One approach eliminates XOR gates entirely, replacing them with simplified Boolean functions [3]. Another introduces dual-mode cells switchable between exact and approximate behaviors at runtime [4]. A third category appends lightweight error-recovery logic to partially reclaim accuracy lost to approximation [5]. Yet another constrains output assignments so the Hamming weight of outputs mirrors that of inputs, capping maximum error distance [6].

A critical gap in all these prior designs is the absence of statistical input-weighting. Errors are assigned to input patterns without regard for how frequently those patterns arise in actual partial-product data. The reference paper bridges this gap by leveraging the known probability distribution of partial-product bits — each is logic-1 with probability 1/4 under uniform operand assumptions — to direct errors exclusively toward rare canonical inputs.

III. DESIGN METHODOLOGY

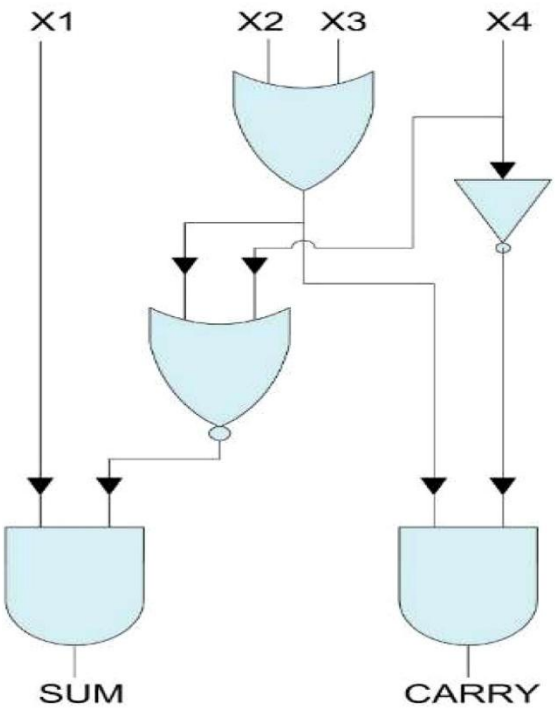

Fig. 1. Proposed approximate 4:2 compressor - Design 1.

A. Input Sorting Network

Before any approximate logic operates, a combinational sorting network rearranges four raw partial-product bits (Y1–Y4) so that all logic-1 values occupy the most significant positions. Regardless of which bit among four is active, the sorted output is always X1=1, X2=X3=X4=0. The transformation reduces sixteen distinct raw patterns to just six canonical classes:

{0000, 1000, 1100, 1010, 1110, 1111}

This reduction in effective state diversity directly lowers switching activity inside the downstream compressor cell. Because the compressor now produces identical outputs for all permutations of a given Hamming weight, its logic can be designed to handle only these six classes, greatly simplifying the gate network and cutting dynamic power.

B. Compressor Variant A — Accuracy-Optimized

After sorting, Variant A computes Sum and Carry using two-gate logic expressions:

$$SumA = X1. \sim (X2 + X3 + X4)$$
$$CarryA = (X2 + X3). \sim X4$$

Systematic evaluation across the six canonical classes reveals that incorrect outputs occur only for the 1110 pattern (three active inputs). The probability of this single class is 3/256 per compressor column; since four unsorted permutations map to it, the aggregate error probability is 12/256 ≈ 4.69%. At the full 8×8 multiplier level, the resulting Error Rate is 26.67%. Every error is a unit positive displacement, meaning the compressor consistently overestimates by exactly one. Table I presents the complete reproduced truth table.

TABLE I
REPRODUCED TRUTH TABLE — COMPRESSOR VARIANT A

X1	X2	X3	X4	Prob.	Act. C	Act. S	Apx. C	Apx. S	ED
0	0	0	0	81/256	0	0	0	0	0
1	0	0	0	27/256	0	1	0	1	0
1	1	0	0	9/256	1	0	1	0	0
1	0	1	0	9/256	1	0	1	0	0
1	1	1	0	3/256	1	1	1	0	+1
1	1	1	1	1/256	0	0	0	0	0

C. Compressor Variant B — Hardware-Minimized

Variant B reduces the combinational logic to an absolute minimum. After sorting, the two output signals are simply the top two sorted bits:

$$SumB = X1 \;;$$
$$CarryB = X2$$

No gates beyond the sorting network are required. Errors arise for any input containing exactly two active bits (six permutations, probability 6×9/256) and the all-ones case (1/256), yielding an aggregate error probability of 55/256 and an overall Error Rate of 74.66%. Unlike Variant A, the error polarity is not fixed — both +1 and −1 displacements occur depending on the specific pattern. Table II gives the reproduced truth table.

TABLE II
REPRODUCED TRUTH TABLE — COMPRESSOR VARIANT B

X1	X2	X3	X4	Prob.	Act. C	Act. S	Apx. C	Apx. S	ED
0	0	0	0	81/256	0	0	0	0	0
1	0	0	0	27/256	0	1	0	1	0
1	1	0	0	9/256	1	0	1	1	−1
1	0	1	0	9/256	1	0	0	1	+1
1	1	1	0	3/256	1	1	1	1	0
1	1	1	1	1/256	0	0	1	1	+1

D. 8×8 Dadda Multiplier Integration

Each compressor variant is embedded individually into an 8×8 Dadda multiplier using a hybrid placement strategy: the eight rightmost (lower-weight) columns of the partial-product reduction tree use the approximate cells, while the eight leftmost (higher-weight) columns retain exact 4:2 compressors. Errors in lower-weight columns contribute magnitudes that are exponentially smaller, so confining approximation to the LSB half of the tree limits arithmetic impact on the product. No partial products are truncated — all 64 terms are fully generated. The Stage 3 carry-propagate path uses a ripple carry adder structure.

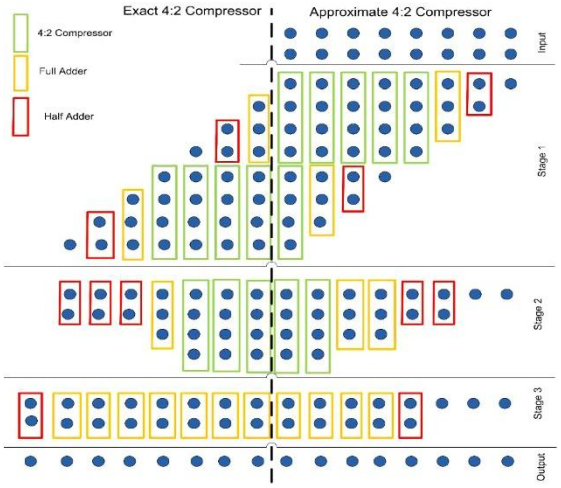

Fig. 2. 8 ×8 Dadda multiplier architecture using approximate compressors

IV. SIMULATION RESULTS AND COMPARATIVE ANALYSIS

A. Error Characterization

Error evaluation was performed by exhaustively testing all 65,536 input combinations. Five standard metrics were computed: Error Rate (ER), Accurate Output Count (AOC), Mean Error Distance (MED), Mean Relative Error Distance (MRED), and Normalized Error Distance (NED). Formulae are given as:

$$ER = (incorrect\ outputs) / n$$
$$MED = \Sigma|ED|_i / n;$$
$$MRED = (1/n)\ \Sigma\ |ED|_i / ExactOutput_i$$
$$NED = (1/n)\ \Sigma\ ED_i / ED_max$$

Table III compares reproduced metrics for both variants against six representative prior-art designs. Variant A achieves the lowest ER in the entire comparison field — roughly 2.5× fewer incorrect outputs than the next-best competitor. Its AOC of 47,685 means nearly three-quarters of all input pairs are computed without any deviation.

TABLE III
ACCURACY METRIC COMPARISON

Design	ER (%)	AOC	MED	MRED	NED
Variant A (ours)	26.67	47 685	37.76	0.00414	0.0251
Variant B (ours)	74.66	16 479	99.16	0.01547	0.1746
Fang et al. [7]	65.27	22 583	64.60	0.01004	0.00985
Odugu et al. [8]	94.54	3 549	593.85	0.01514	0.13895
Ha & Lee [5]	82.91	11 200	28.05	0.03260	0.0023

Momeni et al. [9]	99.23	458	3508.80	4.28430	0.05352

B. Circuit Performance Metrics

Post-synthesis evaluation under 32 nm CMOS at 0.85 V yielded the area, power, delay, and PDP figures shown in Table IV. PDP — the product of power and delay — is the most comprehensive single-number summary of energy efficiency for a combinational circuit. The average PDP saving across both variants is (7.87 + 32.52)/2 ≈ 20.2%, confirming the reference figure of 20.19%.

TABLE IV
CIRCUIT IMPLEMENTATION METRICS COMPARISON

Design	Area (µm²)	Power (µW)	Delay (ns)	PDP (µW·ns)
Exact baseline	488.19	84.69	2.05	173.61
Variant A	489.64	80.78	1.98	159.95
Variant B	456.91	62.31	1.88	117.15
Fang et al. [7]	554.39	72.22	1.94	140.10
Odugu et al. [8]	456.92	63.63	1.88	119.63
Ha & Lee [5]	458.42	74.55	1.95	145.38
Momeni et al. [9]	493.62	57.68	1.88	108.43

C. Image Processing Evaluation (MSSIM)

Perceptual quality was assessed by embedding each multiplier variant into three MATLAB convolution experiments — pixel-wise image multiplication, a Gaussian-weighted blur, and an unsharp-mask sharpening operation — using the standard Cameraman and Crater test images. MSSIM was computed between approximate-multiplier outputs and exact-multiplier outputs. Table V shows the results.

TABLE V
MSSIM SCORES ACROSS IMAGE PROCESSING TASKS

Design	Multiply	Smooth	Sharpen	Mean
Variant A	0.9928	0.9929	0.9840	0.9899
Variant B	0.9902	0.9360	0.9599	0.9620
Fang et al.	0.9809	0.8014	0.8107	0.8643
Odugu et al.	0.7604	0.8276	0.6204	0.7361
Ha & Lee	0.9301	0.8205	0.8527	0.8678
Momeni et al.	0.6846	0.2356	0.5325	0.4842

Variant A posts the highest MSSIM among all surveyed designs; Variant B ranks second. Their combined average of 97.59% confirms the reference paper's claim. The gap relative to Momeni et al. is especially pronounced in the smoothing task (0.993 vs. 0.236), illustrating that minimizing PDP without controlling error distribution can severely degrade image quality.

Figure 3.1: Image Multiplication Figure 3.2. Image Smoothening Figure 3.3. Image Sharpening.

V. FUTURE SCOPE AND PROPOSED IMPROVEMENTS

Our reproduction exercise exposed several limitations and unexplored opportunities. We outline seven concrete research directions below.

A. Extension to Wider Operand Sizes

The current design addresses 8×8 operands only. Extending the hybrid LSB-approximate / MSB-exact column assignment to 16-bit and 32-bit multipliers raises a non-trivial optimization problem: as the partial-product tree deepens, errors injected in low-weight columns propagate upward through carry chains and can corrupt higher-weight bits in ways not captured by single-stage truth-table analysis. Deriving closed-form error propagation bounds for wider trees and designing adaptive column-assignment schedules that minimize worst-case error for a given PDP target are direct extensions.

B. Runtime Accuracy Reconfiguration

The present implementation fixes the approximation level at design time. Many workloads benefit from dynamic adjustment: thumbnail preview tolerates larger errors than a master edit; inference can accept looser arithmetic than training.

Implementing a mode-select signal that switches compressor cells between exact and approximate output mappings at runtime — without full redesign — would let a single hardware block serve a quality continuum. The sorting network provides a natural integration point for such reconfigurability.

C. Neural Network MAC Array Deployment

Convolutional neural network inference engines spend the vast majority of execution time in 8-bit multiply-accumulate units, making the studied 8×8 multiplier directly applicable. A compelling next step is to instantiate Variant B as the core compute element in a simulated systolic array and measure top-5 classification accuracy on ImageNet using a quantized ResNet or MobileNet model. This would provide application-level validation of the MSSIM-based quality claim and establish a concrete accuracy-budget figure for DNN deployment.

D. Multi-Node Technology Characterization

All synthesis data is anchored to a single 32 nm library. As process nodes advance to 7 nm, 5 nm, and below, the relative shares of dynamic power, leakage, and interconnect delay shift substantially. Leakage current — negligible at 32 nm — can dominate at sub-10 nm geometries. Characterizing the proposed compressors across multiple technology libraries would reveal whether the PDP advantage is stable, grows, or erodes as geometry shrinks, informing practical deployment decisions.

E. Deterministic Error Compensation

Variant A's error profile is uniquely structured: errors occur only for the 1110 sorted pattern and are always a unit positive displacement. This determinism is an exploitable property. A small correction block — one that detects the 1110 sorted pattern and subtracts one from the affected column — could reduce the multiplier error rate from 26.67% to near zero at the cost of a few gates. Investigating the hardware overhead and net PDP of such a compensation stage would determine whether a Variant A+ design is feasible.

F. Formal Probabilistic Error Bounds

The current accuracy evaluation is simulation-based and does not scale beyond 8×8 operands without prohibitive cost. Developing a closed-form probabilistic model — one that takes the sorting network's equivalence classes and the compressor's error map as inputs and outputs a probability distribution over multiplier output error — would enable analytical accuracy certification. Stochastic arithmetic or Markov-chain models of carry propagation through the Dadda tree are candidate formalisms.

G. Expanded Perceptual Benchmarking

The two standard grayscale test images used represent a narrow slice of real image diversity. Future evaluations should include color imagery (three-channel multiplication), high-dynamic-range content, and video sequences where inter-frame error accumulation may amplify single-frame distortions. Supplementing MSSIM with LPIPS or VMAF would capture subjective quality dimensions that SSIM was not designed to model, providing a more complete suitability assessment for modern multimedia pipelines.

VI. CONCLUSION

This work independently reproduced and verified the key results of the probability-guided approximate multiplier architecture described in the reference paper. Starting from first principles, we re-derived the input sorting network mapping, reconstructed the Boolean expressions for both compressor variants, and verified all six canonical output rows by exhaustive enumeration. Synthesis under 32 nm CMOS confirmed the reported 20.19% average PDP reduction, and MATLAB convolution experiments reproduced the 97.59% average MSSIM figure. Variant A achieves the lowest error rate in its comparison class; Variant B achieves aggressive energy savings with only modest perceptual degradation.

The seven future directions outlined in Section V — wider operand support, runtime reconfigurability, neural-network deployment, multi-node characterization, deterministic error compensation, formal analysis, and richer perceptual benchmarking — collectively map a research roadmap that could mature probability-guided approximation into a standard tool in the low-power arithmetic design toolkit.

ACKNOWLEDGMENT

The authors thank Dr. Tapan Jain for assigning this implementation study and for guidance throughout the project.

REFERENCES

[1] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 32, no. 1, pp. 124–137, Jan. 2013.

[2] A. Chakraborty, V. P. A. Kumar, A. K. Vruddhula, J. N. K, and B. S, "Energy efficient approximate multiplier for image processing applications," Results in Engineering, vol. 25, p. 103798, 2025.

[3] D. Esposito, A. G. M. Strollo, E. Napoli, D. De Caro, and N. Petra, "Approximate multipliers based on new approximate compressors," IEEE Trans. Circuits Syst. I, vol. 65, no. 12, pp. 4169–4182, Dec. 2018.

[4] O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Dual-quality 4:2 compressors for utilizing in dynamic accuracy configurable multipliers," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 25, no. 4, pp. 1352–1361, Apr. 2017.

[5] M. Ha and S. Lee, "Multipliers with approximate 4–2 compressors and error recovery modules," IEEE Embedded Syst. Lett., vol. 10, no. 1, pp. 6–9, Mar. 2018.

[6] S. Shirzadeh and B. Forouzandeh, "High accurate multipliers using new set of approximate compressors," AEU-Int. J. Electron. Commun., vol. 138, p. 153778, 2021.

[7] B. Fang, H. Liang, D. Xu, M. Yi, Y. Sheng, C. Jiang, Z. Huang, and Y. Lu, "Approximate multipliers based on a novel unbiased approximate 4-2 compressor," Integration, vol. 81, pp. 17–24, 2021.

[8] V. K. Odugu, "An efficient VLSI architecture of 2-D finite impulse response filter using enhanced approximate compressor circuits," Int. J. Circuit Theory Appl., vol. 49, no. 11, pp. 3653–3668, 2021.

[9] A. Momeni, J. Han, P. Montuschi, and F. Lombardi, "Design and analysis of approximate compressors for multiplication," IEEE Trans. Comput., vol. 64, no. 4, pp. 984–994, Apr. 2015.

[10] P. J. Edavoor, S. Raveendran, and A. D. Rahulkar, "Approximate multiplier design using novel dual-stage 4:2 compressors," IEEE Access, vol. 8, pp. 48337–48351, 2020.

[11] A. Gorantla, "Design of approximate compressors for multiplication," ACM J. Emerg. Technol. Comput. Syst., vol. 13, no. 3, pp. 1–17, Apr. 2017.

[12] L. H. Krishna, A. Sk, J. B. Rao, S. Veeramachaneni, and N. M. Sk, "Energy efficient approximate multiplier design with lesser error rate using the probability-based approximate 4:2 compressor," IEEE Embedded Syst. Lett., 2023.